Arbres de suffixes pour recherche de motifs

18 septembre 2023 09:04

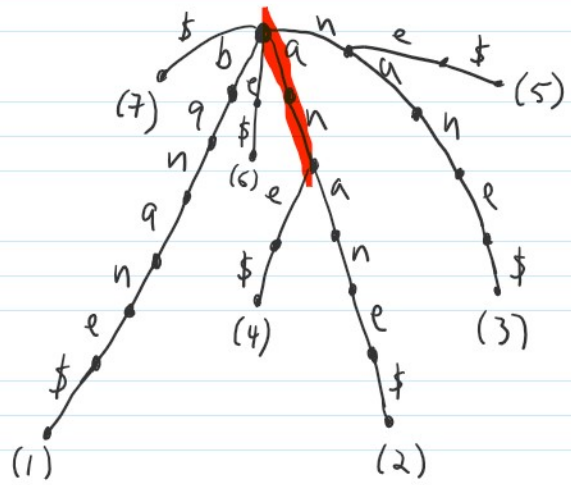
Struct. de données utile pour plusieurs pbs de recherche de séquences

Arbustes de suffixes (trie)

- arbre enraciné
- chaque arête a un symbole
- pour tout noeud, les arêtes enfantes ont un symbole différent
- tout chemin racine-feuille correspond à un suffixe de notre séquence S
  suffixe = $S[i..|S|]$
- on suppose que S termine par un symbole unique $

ex: S = banane $

1 2 3 4 5 6 7

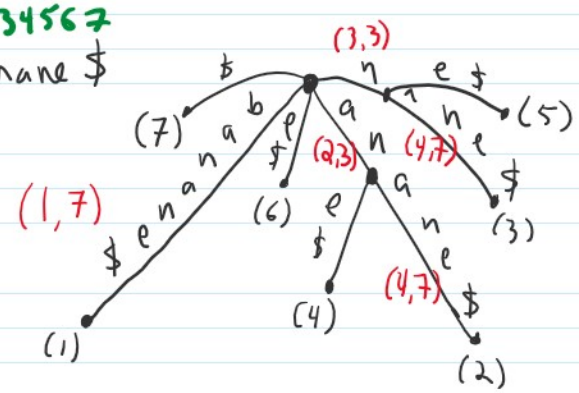


Arbre de suffixes

- arbuste dans lequel on a contracté les noeuds avec un seul enfant
- arêtes étiquetées par des sous-chaînes

1 2 3 4 5 6 7

S = banane $



En réalité, chaque arête est étiquetée par 2 entiers (i, j), telles que $S[i..j]$ est la chaîne à stocker sur l'arête.

- Quel est l'espace en mémoire requis pour stocker un arbre de suffixes? (en fct de $n = |S|$)

  → n feuilles
  → $\leq n$ noeuds internes (exercice)
  → $\leq 2n$ noeuds
  → $\leq 2n$ arêtes

  Stocker l'arbre : $O(n)$ espace

  Espace par feuille : $O(1)$ pour stocker pos. de S (en fait, $\log n$)
  Espace par branche : $O(2) = O(1)$ pour paire de pos. de S

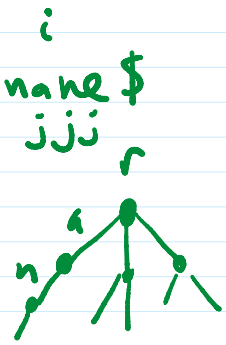
  → Espace total : $O(n)$

Temps de construction :
1) Construire arbuste $O(n^2)$
2) Contracter les noeuds avec 1 enfant $O(n^2)$

Il existe des algos qui construisent les arbres en $O(n)$
- Algo de Ukkonen (1980)
- Algo de Farach-Colton (1990)

Construire arbuste

$S =$ banane\$



Entrée : séq. S

```
r = new Noeud()

pour i = 1 .. |S|
    cur = r
    pour j = i .. |S|
        si cur a un enfant étiqueté S[j]
```

```
pour j = i .. |S|
    si cur a un enfant étiqueté S[j]
        cur = l'enfant étiqueté S[j]
    sinon
        v = new Noeud()
        étiqueter (cur, v) par S[j]
        cur = v
cur.position = i
```